

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Artificial Intelligence Application Development
Module 1
Practical 7

Roll no. : T022	Name : Pratik Mishra
Class : TYIT	Batch : 1
Date of assignment : 08/08/2026	Date of assignment : 08/08/2026

6. Implement Decision Tree regression on a regression dataset. Code & Output:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor

# Load Iris dataset
data = pd.read_csv("iris.csv")

# Input and output
X = data[["petal_length"]]
y = data["sepal_length"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create model
model = DecisionTreeRegressor()

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
new_data = pd.DataFrame([[5.0]], columns=["petal_length"])
prediction = model.predict(new_data)
```

```
print("Predicted Sepal Length for Petal Length 5.0:", prediction[0])
```

✓ 0.0s

Training Accuracy: 90.6283037982957
Testing Accuracy: 61.94673364225676
Predicted Sepal Length for Petal Length 5.0: 5.7

7. Implement k-Nearest Neighbors(kNN)

classifier. Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create kNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Prediction
prediction = model.predict([X_test[0]])

print("Predicted Class:", prediction[0])
```

✓ 0.0s

Output:

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Class: 0
```

8. Implement multiclass classification using Logistic Regression or Decision Tree.

Code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Create Decision Tree model
model = DecisionTreeClassifier()

# Train model
model.fit(X_train, y_train)

# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train) * 100)
print("Testing Accuracy:", model.score(X_test, y_test) * 100)

# Predict one sample
prediction = model.predict([X_test[0]])

print("Predicted Class:", prediction[0])
```

✓ 0.0s

Output:

```
Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
```

Task: Perform Logistic Regression, Decision Tree, and kNN on the same dataset. Compare the Results.

Code:

```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

# Load Wine dataset
wine = load_wine()

X = wine.data
y = wine.target

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)

# Logistic Regression
logistic = LogisticRegression(max_iter=10000)
logistic.fit(X_train, y_train)

logistic_accuracy = logistic.score(X_test, y_test) * 100

# Decision Tree
tree = DecisionTreeClassifier(random_state=1)
tree.fit(X_train, y_train)

tree_accuracy = tree.score(X_test, y_test) * 100

# kNN
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

knn_accuracy = knn.score(X_test, y_test) * 100

# Display results
print("Logistic Regression Accuracy:", logistic_accuracy)
print("Decision Tree Accuracy:", tree_accuracy)
print("kNN Accuracy:", knn_accuracy)

# Compare results
if logistic_accuracy >= tree_accuracy and logistic_accuracy >= knn_accuracy:
    print("Best Model: Logistic Regression")
elif tree_accuracy >= logistic_accuracy and tree_accuracy >= knn_accuracy:
    print("Best Model: Decision Tree")
else:
    print("Best Model: kNN")

```

✓ 2.3s

Output:

```

Logistic Regression Accuracy: 96.29629629629629
Decision Tree Accuracy: 94.44444444444444
kNN Accuracy: 74.07407407407408
Best Model: Logistic Regression

```